



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Compiladores

Árboles de sintaxis abstracta

Práctica 4

Presenta:
Ramírez Juárez María Fernanda - 321204747
Rojo Peña Manuel Ianluck - 118005762

Profesor:
Pablo Martínez Yessica Janeth

Ayudante de laboratorio:
Salazar González Pedro Yamil

Grupo: 7010, 2027-1

Fecha de entrega:
2 de octubre, 2026

Objetivo

En esta práctica agregamos al compilador de MINIC la primera etapa del análisis semántico. Hasta la Práctica 4 el compilador respondía *¿qué elementos forman el programa?* y *¿está sintácticamente bien escrito?*; ahora también responde *¿tiene sentido lo que está escrito?*. Un programa puede cumplir la gramática y aun así usar una variable que nunca se declaró, o declarar dos veces el mismo identificador en el mismo ámbito.

El analizador semántico no vuelve a leer el archivo fuente ni a pedir tokens: recorre el AST que produjo el parser. En esta entrega no verificamos tipos de expresiones, compatibilidad de asignaciones ni condiciones booleanas, y tampoco generamos código. Concretamente, el objetivo es:

1. Diseñar una tabla de símbolos que represente cada variable declarada
2. Implementar ámbitos anidados, con un ámbito global y uno nuevo por cada bloque
3. Verificar que toda variable se declare antes de usarse
4. Prohibir la redeclaración dentro del mismo ámbito y permitir el *shadowing* entre ámbitos distintos
5. Reportar múltiples errores semánticos con línea y columna, sin detenerse en el primero
6. Distinguir los errores semánticos del programa fuente de los fallos internos (memoria)
7. Definir un único propietario para cada recurso y liberar la tabla sin fugas

Evolución respecto de la Práctica 4

Módulos modificados o agregados

- `symbol.h` y `symbol.c`: módulo nuevo con el tipo `Type`, la estructura `Symbol` y sus funciones de creación y destrucción.
- `scope.h` y `scope.c`: módulo nuevo con la estructura `Scope`, una tabla hash dinámica por ámbito y las operaciones de búsqueda e inserción dentro de un ámbito.
- `symbol_table.h` y `symbol_table.c`: módulo nuevo con la pila de ámbitos, la entrada y salida de ámbitos, la búsqueda local y visible, la inserción y la destrucción.
- `semantic.h` y `semantic.c`: módulo nuevo con el analizador, el recorrido del AST y los diagnósticos.
- `main.c`: después del parser ejecuta el análisis semántico y cambia la salida normal.
- `Makefile`: incorpora los archivos nuevos y las pruebas de esta práctica.
- `lexer`, `token`, `keywords`, `buffer_manage`, `parser.c`, `grammar.c`, `ast.c` y `ast.h` se conservan sin cambios.

Arquitectura actualizada

```
main
 |
 v
Lexer ----> Parser ----> AST
                          |
                          v
                  Semantic Analyzer
                   |-- recorrido del arbol
                   |-- manejo de scopes
                   |-- insercion y busqueda de simbolos
                   '-- diagnosticos
                          |
                          v
                    Symbol Table
                          |
                          v
                Stack de Scopes (global -> ... -> actual)
```

Cada módulo tiene una responsabilidad separada: el lexer reconoce tokens, el parser construye el AST, el analizador semántico decide si el árbol tiene sentido, y la tabla de símbolos solo almacena símbolos y administra ámbitos. La tabla no conoce las reglas del lenguaje: no imprime diagnósticos ni recorre el AST.

Cambio de salida en `main.c`

A partir de esta práctica la salida normal es distinta:

- Programa válido: `stdout` contiene exactamente `Programa semanticamente correcto.` y el código de salida es `0`.
- Errores léxicos, sintácticos o semánticos: `stdout` queda vacío, los diagnósticos van a `stderr` y el código de salida es `1`.
- Uso incorrecto, error de entrada y salida o fallo interno de memoria: se informa por `stderr` y el código de salida es `2`.

Ya no se imprimen el AST ni el mensaje `Programa sintacticamente correcto.` en la ejecución normal. Para no perder la herramienta de depuración, `ast_print` se conserva y se activa con la bandera `-ast`, que reproduce exactamente la salida de la Práctica 4 y no ejecuta el análisis semántico. Así las pruebas de las prácticas anteriores siguen siendo utilizables.

```
1     result = parser_parse_program(&parser, &root);
2
3     switch (result) {
4     case PARSE_OK:
5         if (show_ast) { /* modo depuracion: comportamiento de la P4 */
6             printf("AST:\n");
7             ast_print(root);
```

```
            printf("Programa sintacticamente correcto.\n");
            exit_code = MINIC_EXIT_OK;
        } else {
            exit_code = analyze(root);
        }
        break;
    case PARSE_INVALID:
        exit_code = MINIC_EXIT_INVALID; /* no se corre el semantico */
        break;
    default:
        exit_code = MINIC_EXIT_INTERNAL;
        break;
    }

    ast_destroy(root);
    parser_destroy(&parser);
    lexer_destroy(&lexer);
```

Código 1: Flujo de `compile` en `main.c` (resumido).

El orden de liberación es siempre el mismo: el analizador se destruye dentro de `analyze`, y después se liberan el AST, el parser y el lexer. Como el analizador no es propietario de los nodos del árbol, el orden entre el analizador y el AST no puede producir accesos inválidos.

Diseño de la tabla de símbolos

Representación de un símbolo

Cada variable declarada se representa con la estructura `Symbol`:

```
typedef enum { TYPE_INT, TYPE_BOOL, TYPE_ERROR, TYPE_UNKNOWN } Type;

typedef struct Symbol {
    char *name;               /* copia propia */
    Type type;
    int scope_level;
    int line;
    int column;
    ASTNode *declaration;     /* NO es propiedad del simbolo */
    struct Symbol *next;      /* encadenamiento en el bucket */
} Symbol;
```

Código 2: Estructura `Symbol` de `symbol.h`.

El símbolo conserva el nombre, el tipo declarado, el nivel de ámbito, la posición de la declaración y una referencia al nodo del AST donde se declaró. En esta práctica solo se registran `TYPE_INT` y `TYPE_BOOL`; `TYPE_ERROR` y `TYPE_UNKNOWN` quedan reservados para la verificación de tipos. El campo `next` se agregó a la estructura recomendada para poder encadenar los símbolos que caen en el mismo bucket de la tabla hash.

- `symbol_create` reserva el símbolo y copia el nombre. Si cualquiera de las dos reservas falla, libera lo que ya había reservado y devuelve `NULL`. El nombre es una copia propia, por lo que no depende de la memoria del lexer ni del parser.

- `symbol_destroy` libera el nombre y la estructura, acepta `NULL` y **nunca** libera el nodo de declaración, porque ese nodo pertenece al AST.

La posición del símbolo es la que ya fija el AST de la Práctica 4: la declaración usa la posición del *identificador* (no la de `int` o `bool`), de modo que `int x;` produce la posición `[1:4]`. Esto coincide con los diagnósticos que pide el enunciado.

Estructura de un ámbito

```
typedef struct Scope {
    Symbol **buckets;
    size_t bucket_count;
    size_t symbol_count;
    int level;
    struct Scope *parent;
} Scope;
```

Código 3: Estructura `Scope` de `scope.h`.

Cada ámbito guarda únicamente los símbolos declarados directamente en él, su nivel (el global es `0` y cada bloque suma uno respecto de su padre) y un apuntador a su ámbito exterior. Para almacenar los símbolos elegimos una tabla hash con encadenamiento:

- La función de dispersión es `djb2` sobre el nombre del identificador.
- Cada ámbito comienza con 16 buckets.
- Antes de insertar se verifica el factor de carga: si `(symbol_count + 1) * 4 > bucket_count * 3`, es decir, si superaría 0.75, el arreglo de buckets duplica su tamaño y todos los símbolos se redistribuyen (*rehash*).
- Por tanto no existe un límite fijo de símbolos por ámbito, y las colisiones se resuelven con la lista encadenada de cada bucket.

```
static int
scope_grow(Scope *scope)
{
    size_t new_count = scope->bucket_count * 2;
    size_t i;
    Symbol **nb = calloc(new_count, sizeof *nb);
    if (nb == NULL)
        return 0;
    for (i = 0; i < scope->bucket_count; i++) {
        Symbol *s = scope->buckets[i];
        while (s != NULL) {
            Symbol *next = s->next;
            size_t idx = hash_name(s->name) % new_count;
            s->next = nb[idx];
            nb[idx] = s;
            s = next;
        }
    }
```

```
    free(scope->buckets);
    scope->buckets = nb;
    scope->bucket_count = new_count;
    return 1;
}
```

Código 4: Crecimiento de la tabla hash en `scope.c`.

El nuevo arreglo se reserva primero y el anterior solo se libera cuando la redistribución terminó, de modo que si la reserva falla la tabla original queda intacta. En ese caso `scope_insert` devuelve un fallo interno y no un duplicado.

La tabla de símbolos y la pila de ámbitos

```
typedef struct {
    Scope *current_scope;
    size_t scope_count;
} SymbolTable;
```

Código 5: Estructura `SymbolTable` de `symbol_table.h`.

La pila de ámbitos se representa con la cadena de apuntadores `parent`: `current_scope` es la cima y el ámbito global es la base (su `parent` es `NULL`). No se necesita un arreglo aparte, y entrar o salir de un ámbito solo mueve un apuntador.

Códigos de resultado

Las operaciones que pueden fallar distinguen entre éxito, duplicado y fallo interno:

```
typedef enum {
    ST_OK,              /* operacion correcta */
    ST_DUPLICATE,       /* el nombre ya existe en el ambito actual */
    ST_INTERNAL_ERROR,  /* fallo de memoria o estado invalido */
    ST_AT_GLOBAL        /* se intento salir del ambito global */
} SymbolTableResult;

typedef enum {
    SEM_OK,             /* sin errores semanticos */
    SEM_ERRORS,         /* errores semanticos en el programa fuente */
    SEM_INTERNAL_ERROR  /* fallo interno; analisis detenido */
} SemanticResult;
```

Código 6: Códigos de resultado de la tabla y del analizador.

Esta separación es importante: un duplicado es un error del programa fuente que se puede recuperar y acumular, mientras que un fallo de memoria es un error interno que detiene el análisis. Con un único código de error no habría forma de evitar que un `malloc` fallido se reportara como una redeclaración.

Operaciones de la tabla

- `symbol_table_init`: deja la tabla en un estado seguro (sin ámbito y con contador en cero) antes de reservar el ámbito global. Si la reserva falla, la tabla queda en un estado que `destroy` puede procesar sin riesgo.
- `symbol_table_enter_scope`: crea un ámbito hijo del actual y solo lo convierte en el ámbito vigente si la reserva terminó bien. Si falla, `current_scope` no cambia.
- `symbol_table_exit_scope`: restaura el ámbito padre y libera el ámbito abandonado. Rechaza la salida del ámbito global devolviendo `ST_AT_GLOBAL`.
- `symbol_table_lookup_current`: busca únicamente en el ámbito actual.
- `symbol_table_lookup`: búsqueda visible, del ámbito actual hacia el global.
- `symbol_table_insert`: inserta solo en el ámbito actual y devuelve `ST_OK`, `ST_DUPLICATE` o `ST_INTERNAL_ERROR`.
- `symbol_table_destroy`: libera todos los ámbitos que todavía pertenecen a la tabla, desde el actual hasta el global, y es segura tras una inicialización incompleta.

Búsqueda local y búsqueda visible

Son dos operaciones distintas con dos propósitos distintos:

- La **búsqueda local** (`lookup_current`) revisa solo el ámbito actual y no consulta los exteriores. Se usa al *declarar*, para detectar redeclaraciones.
- La **búsqueda visible** (`lookup`) revisa el ámbito actual y, si no encuentra el nombre, sigue con el padre hasta llegar al global. Devuelve `NULL` si el identificador no existe en ninguno. Se usa al *usar* un identificador, para validar que esté declarado.

```
Symbol *
symbol_table_lookup(SymbolTable *table, const char *name)
{
    Scope *s;
    if (table == NULL)
        return NULL;
    for (s = table->current_scope; s != NULL; s = s->parent) {
        Symbol *found = scope_lookup(s, name);
        if (found != NULL)
            return found;
    }
    return NULL;
}
```

Código 7: Búsqueda visible en `symbol_table.c`.

Como la búsqueda avanza del ámbito más interno al más externo, el primer símbolo encontrado es siempre el de la declaración más cercana. Eso es lo que implementa el *shadowing*.

Propiedad y liberación de memoria

Definimos un único propietario para cada recurso:

- **El símbolo** es propietario de la copia de su nombre. No es propietario del nodo de declaración del AST: `symbol_destroy` nunca lo libera.
- **El ámbito** es propietario de sus símbolos y de su arreglo de buckets.
- **La tabla** es propietaria de los ámbitos que forman la pila vigente.
- **El analizador** es propietario de la tabla, pero *no* del AST: el árbol sigue perteneciendo a `main.c`, que lo libera con `ast_destroy`.
- **Inserción:** la tabla adquiere el símbolo únicamente si `symbol_table_insert` devuelve `ST_OK`. En cualquier otro caso, quien llamó conserva la responsabilidad de liberarlo.

Estrategia de liberación de ámbitos

Elegimos la **liberación inmediata**: al salir de un ámbito se destruyen sus símbolos y su arreglo de buckets. Esta estrategia es válida en esta práctica porque el AST no guarda referencias a los símbolos (`Symbol *`); la asociación entre usos y declaraciones se resuelve durante el recorrido y no se conserva después. Además, deja la tabla con tamaño proporcional al anidamiento actual y no al del programa completo.

Si en la Práctica 6 el verificador de tipos anotara el AST con apuntadores a símbolos, esta estrategia tendría que cambiar: los ámbitos retirados deberían moverse a una lista de ámbitos conservados y liberarse hasta `symbol_table_destroy`, para no dejar apuntadores colgantes. Dejamos esta decisión documentada en un comentario de `symbol_table.c`.

```
SymbolTableResult
symbol_table_exit_scope(SymbolTable *table)
{
    Scope *old;
    if (table == NULL || table->current_scope == NULL)
        return ST_INTERNAL_ERROR;
    if (table->current_scope->parent == NULL)
        return ST_AT_GLOBAL;
    old = table->current_scope;
    table->current_scope = old->parent;
    table->scope_count--;
    scope_destroy(old);
    return ST_OK;
}
```

Código 8: Salida de un ámbito.

Recorrido del AST

Estructura del analizador

```c
typedef struct {
    SymbolTable symbol_table;
    int error_count;
    int had_internal_error;
} SemanticAnalyzer;

int semantic_analyzer_init(SemanticAnalyzer *analyzer);
SemanticResult semantic_analyze(SemanticAnalyzer *analyzer,
                                ASTNode *program);
void semantic_analyzer_destroy(SemanticAnalyzer *analyzer);
```

Código 9: Estructura `SemanticAnalyzer` de `semantic.h`.

El analizador contabiliza los errores semánticos en `error_count` y marca los fallos internos con `had_internal_error`, de modo que ambos se pueden distinguir. `semantic_analyze` devuelve `SEM_OK` si no hubo errores, `SEM_ERRORS` si el programa fuente tiene errores semánticos y `SEM_INTERNAL_ERROR` si ocurrió un fallo interno.

Funciones de visita

El recorrido es recursivo y está dividido en funciones pequeñas, una por categoría de nodo, en lugar de una sola función extensa. Todas son `static` dentro de `semantic.c`:

- `visit_program`: recorre las sentencias en el orden en que aparecen y se detiene si hubo un fallo interno. El ámbito global ya existe desde `semantic_analyzer_init`.
- `visit_statement`: despacha según el tipo de nodo hacia la función correspondiente.
- `visit_block`: entra a un ámbito, recorre las sentencias y sale.
- `visit_variable_declaration`: registra la variable y luego visita el inicializador.
- `visit_assignment`: valida el destino y visita el valor.
- `visit_print_statement`, `visit_if_statement` y `visit_while_statement`: visitan sus expresiones y sentencias hijas.
- `visit_expression`: busca los identificadores dentro de expresiones binarias y unarias; los literales no requieren búsqueda.

Las declaraciones no se insertan por adelantado: se procesan en el orden del programa, de modo que una declaración solo es visible después de haber sido recorrida. Por eso `x = 1; int x;` produce un error.

Declaraciones

```c
    Symbol *prev = symbol_table_lookup_current(&a->symbol_table, name);

    if (prev != NULL) {
        /* No se sobrescribe el simbolo original. */
        report_redeclaration(a, prev, node->line, node->column);
```

```
    } else {
        Symbol *sym = symbol_create(name, type_from_ast(...),
            a->symbol_table.current_scope->level,
            node->line, node->column, node);
        if (sym == NULL) { report_internal_error(a); return; }
        r = symbol_table_insert(&a->symbol_table, sym);
        if (r != ST_OK) {          /* la tabla no adquirio el simbolo */
            symbol_destroy(sym);
            if (r == ST_INTERNAL_ERROR) {
                report_internal_error(a);
                return;
            }
        }
    }
    /* El simbolo ya es visible: int x = x; es valido. */
    visit_expression(a, node->data.variable_declaration.initializer);
```

Código 10: `visit_variable_declaration` (resumida).

Las decisiones de diseño relevantes son:

- La redeclaración se detecta con la búsqueda *local*, por lo que declarar un nombre que existe solo en un ámbito exterior es válido (shadowing).
- Ante una redeclaración, el símbolo original no se reemplaza. Esto es importante para la verificación de tipos de la siguiente práctica, y lo comprobamos con una prueba propia: en `int x; bool x; int x;` ambos errores señalan la declaración original en `[1:4]`; si el símbolo se hubiera sobrescrito, el segundo error apuntaría a `[2:5]`.
- El símbolo se inserta *antes* de visitar el inicializador. Siguiendo la regla de esta práctica ("la variable es visible a partir del momento en que su declaración fue registrada"), `int x = x;` no produce un error de variable no declarada. No se evalúa si el valor está inicializado.
- Aun cuando la declaración sea una redeclaración, el inicializador se sigue visitando para poder detectar otros errores en la misma línea.
- Si un símbolo recién creado no puede insertarse, la función que lo creó lo libera, porque la tabla solo adquiere símbolos insertados con `ST_OK`.

Asignaciones, identificadores y expresiones

Una asignación busca su destino con la búsqueda visible y reporta un error si no existe, y después recorre la expresión del valor. Como el nodo de asignación ya guarda la posición del identificador destino, `x = 5;` se reporta en `[1:0]`. Cada nodo identificador que representa un uso se valida con una búsqueda visible; los identificadores que forman parte de una declaración no se validan como usos.

Las expresiones se recorren solo para localizar identificadores. No se infiere ningún tipo ni se comprueba la compatibilidad entre el destino y el valor, tal como pide el enunciado: esa verificación corresponde a la siguiente práctica.

Bloques

```c
static void
visit_block(SemanticAnalyzer *a, ASTNode *node)
{
    size_t i;

    if (symbol_table_enter_scope(&a->symbol_table) != ST_OK) {
        report_internal_error(a);
        return;                    /* no se entro: no se sale */
    }
    for (i = 0; i < node->data.block.statements.count; i++) {
        if (a->had_internal_error)
            break;
        visit_statement(a, node->data.block.statements.items[i]);
    }
    if (symbol_table_exit_scope(&a->symbol_table) != ST_OK)
        report_internal_error(a);
}
```

Código 11: `visit_block`.

La salida del ámbito se realiza siempre que la entrada tuvo éxito, incluso si se detectaron errores semánticos dentro del bloque. Si la entrada falla, no se llama a la salida, porque no hay ámbito que abandonar. Si ocurre un fallo interno dentro del bloque, el recorrido se detiene, el ámbito se abandona y el fallo se propaga.

Condicionales y ciclos

Para `if` se recorren la condición, la rama verdadera y la rama falsa cuando existe; para `while`, la condición y el cuerpo. No se comprueba que la condición sea booleana. Solo los nodos `Block` crean ámbitos: una rama que es una sola sentencia sin llaves, como en `if (c) x = 1; else x = 2;`, se recorre en el ámbito vigente.

Diagnósticos y recuperación

Formato de los errores

Los errores semánticos se envían a `stderr` con la categoría, la línea, la columna y una descripción que incluye el nombre del identificador. La columna usa la convención del lexer (desde 0), igual que los errores léxicos y sintácticos.

- **Variable no declarada:**

  ```
  Error semantico [1:0]:
  la variable 'x' no ha sido declarada.
  ```

- **Redeclaración,** con referencia a la declaración anterior:

```
Error semantico [2:5]:
la variable 'x' ya fue declarada
en este ambito.
Declaracion anterior [1:4].
```

Acumulación de errores

A diferencia de algunos errores sintácticos, un error semántico no impide seguir recorriendo el árbol. El analizador incrementa `error_count`, reporta el diagnóstico y continúa, por lo que detecta varios errores en un mismo programa. Por ejemplo, para

```
x = 1;
y = 2;
int z;
int z;
print(w);
```

se reportan cuatro errores: `x` y `y` no declaradas, la redeclaración de `z` y `w` no declarada. El orden de los diagnósticos es el del recorrido, que coincide con el del programa.

Errores internos

Los fallos de memoria al reservar símbolos, cadenas o ámbitos son errores internos, no errores semánticos. `report_internal_error` imprime un diagnóstico por `stderr` y activa `had_internal_error`; los recorridos de programa y bloque revisan esa bandera antes de cada sentencia y se detienen de forma controlada. El resultado es `SEM_INTERNAL_ERROR`, el código de salida es `2` y los recursos adquiridos se liberan en `semantic_analyzer_destroy`. Nunca se informa una falla de memoria como redeclaración, porque `ST_DUPLICATE` y `ST_INTERNAL_ERROR` son resultados distintos.

Resultados y pruebas

Cambios en el `Makefile`

Respecto de la Práctica 4, el `Makefile` cambió en los siguientes puntos:

- Se agregaron los archivos `src/semantic/symbol.c`, `scope.c`, `symbol_table.c` y `semantic.c` a la variable `SOURCES`, y `include/ast/*.h` e `include/semantic/*.h` a `HEADERS`, de modo que `make` recompile el ejecutable cuando cambia cualquiera de ellos.
- Se agregaron las variables que indican la ubicación de las pruebas públicas y propias de esta práctica:

```
PUBLIC_P5_INPUT_DIR = tests/public/p5/inputs
PUBLIC_P5_EXPECTED_DIR = tests/public/p5/expected

UNIT_SEMANTIC_INPUT_DIR = tests/unit_tests/semantic/inputs
UNIT_SEMANTIC_EXPECTED_DIR = tests/unit_tests/semantic/expected
```

- Se agregó el objetivo `test-unit-semantic`, que usa la plantilla `run_test_dir` y se incluyó en `test-unit`. La plantilla compara siempre `stdout` con el archivo `.out` y, cuando existen, también `stderr` con el `.err` y el código de salida con el `.code`.

```
test-unit-semantic: $(TARGET)
$(call run_test_dir,$(UNIT_SEMANTIC_INPUT_DIR),
  $(UNIT_SEMANTIC_EXPECTED_DIR),,unit-semantic)
```

- Se agregó el objetivo `test-p5` para las pruebas públicas de la práctica, incluido en `test-cli`.
- Como la salida normal cambió, `test-unit-parser` y `test-unit-ast` ahora invocan el ejecutable con la bandera `-ast`, y los scripts de las pruebas públicas de las Prácticas 3 y 4 también. De esta forma conservan sus archivos esperados sin modificarlos.

```
test-unit-ast: $(TARGET)
$(call run_test_dir,$(UNIT_AST_INPUT_DIR),
  $(UNIT_AST_EXPECTED_DIR),--ast,unit-ast)
```

Pruebas públicas de la Práctica 5

Al ejecutar las pruebas públicas proporcionadas, las 24 comprobaciones automáticas fueron aprobadas.

```
sh tests/public/p5/run_public_tests.sh ./minic
[OK automatico] p01_empty
[OK automatico] p02_literals
[OK automatico] p03_declarations
[OK automatico] p04_valid_use
[OK automatico] p05_undeclared_assignment
[OK automatico] p06_use_before_declaration
[OK automatico] p07_global_redeclaration
[OK automatico] p08_local_redeclaration
[OK automatico] p09_nested_visibility
[OK automatico] p10_out_of_scope
[OK automatico] p11_shadowing
[OK automatico] p12_sibling_scopes
[OK automatico] p13_sibling_visibility
[OK automatico] p14_initializer_use
[OK automatico] p15_self_initializer
[OK automatico] p16_multiple_errors
[OK automatico] p17_condition_uses
[OK automatico] p18_if_else
[OK automatico] p19_while
[OK automatico] p20_types_deferred
[OK automatico] p21_syntax_error
[OK automatico] p22_lexical_error
[OK automatico] p23_expression_uses
```

```
[OK automatico] p24_empty_blocks

Comprobaciones automaticas: 24 aprobadas, 0 fallidas.
Casos que requieren revisar diagnosticos: 12.
```

El script marca doce casos como *REVISAR*, pues sus diagnósticos deben compararse contra `expected/CASOS.md`. Revisamos manualmente cada uno y los mensajes coinciden. Por ejemplo, en `p07_global_redeclaration` el diagnóstico es:

```
Error semantico [2:5]:
la variable 'x' ya fue declarada
en este ambito.
Declaracion anterior [1:4].
```

y en `p16_multiple_errors` se reportan, en orden, los cuatro errores esperados (`[1:0]`, `[2:0]`, `[4:4]` con declaración anterior en `[3:4]`, y `[5:6]`). Los casos `p21_syntax_error` y `p22_lexical_error` comprueban que, ante un error sintáctico o léxico, el análisis semántico no se ejecuta y se conserva el diagnóstico de la etapa correspondiente.

Pruebas propias del analizador semántico

Además de las pruebas públicas, diseñamos 40 pruebas propias, con el mismo esquema de las pruebas del AST: cada programa `.mc` tiene su salida estándar esperada (`.out`), sus diagnósticos esperados (`.err`) y su código de salida (`.code`). Se organizaron en `tests/unit_tests/semantic/` con los prefijos `ut_sem_`.

Para no calcular los archivos esperados a mano, escribimos un modelo de referencia independiente de la implementación en C (un analizador léxico y un recorrido con pila de ámbitos) y generamos los `.err` con él. Antes de usarlo lo validamos contra seis casos de los ejemplos públicos (`p07`, `p08`, `p14`, `p16`, `p17` y `p23`), con resultados idénticos. De este modo las pruebas comparan la implementación en C contra un modelo distinto.

Tema	**Casos**
Básicos	`empty`, `no_declarations`, `one_global`, `multiple_globals`
Declaración y uso	`use_after_declaration`, `use_before_declaration`, `undeclared_completely`, `assignment_target`, `expression_use`, `print_use`, `condition_use`, `statement_order`
Redeclaración	`global_redeclaration`, `block_redeclaration`, `redeclaration_other_type`, `redeclaration_keeps_original`, `redeclaration_initializer`, `long_identifier_redecl`
Shadowing y visibilidad	`shadow_global`, `shadow_multilevel`, `outer_access`, `global_from_nested`, `initializer_uses`, `self_initializer`
Salida de ámbito	`out_of_scope`, `out_of_scope_nested`, `sibling_visibility`, `while_local_escape`
Bloques y control	`empty_blocks`, `sibling_blocks`, `if_scopes`, `if_else_valid`, `while_locals`, `if_without_blocks_ok`, `deep_nesting`
Recuperación	`multiple_errors`, `errors_inside_blocks`
Estrés	`many_symbols`, `many_symbols_redecl`, `long_identifiers`

Table 1: Casos propios de `tests/unit_tests/semantic/inputs`, todos con prefijo `ut_sem_`.

Algunos casos merecen explicación:

- `redeclaration_keeps_original` comprueba que una redeclaración no reemplaza al símbolo original: con `int x; bool x; int x;`, ambos errores deben apuntar a la declaración anterior `[1:4]`.
- `many_symbols` declara 300 variables globales y vuelve a declarar la mitad en un bloque, lo que obliga a la tabla hash a hacer varios *rehash* y a resolver colisiones. `many_symbols_redecl` agrega una redeclaración de `v150` y verifica que la tabla la detecte entre cientos de símbolos.
- `long_identifiers` usa identificadores de 100 caracteres, para comprobar que el nombre se copia completo.
- `statement_order` verifica que las sentencias se procesan en orden: usar una variable antes de su declaración es un error, aunque la declaración aparezca después.
- `errors_inside_blocks` y `multiple_errors` verifican que el análisis continúa tras el primer error y que el ámbito se abandona correctamente aunque haya errores dentro.

Ejecución de las pruebas

El sistema ejecuta cada archivo `.mc` y compara la salida estándar, los diagnósticos y el código de salida contra los archivos esperados. Una prueba se considera correcta solo si los tres coinciden.

`[PEGAR AQUI LA SALIDA DE: make test-unit-semantic]`

Pruebas de memoria

Verificamos la implementación con `valgrind` sobre programas válidos, programas con errores semánticos, y sobre los casos de estrés:

```
valgrind --leak-check=full --show-leak-kinds=all \
         ./minic programa.mc
```

`[PEGAR AQUI EL RESUMEN DE VALGRIND (HEAP SUMMARY / ERROR SUMMARY)]`

Las condiciones que se comprobaron fueron la ausencia de fugas al crear símbolos y ámbitos, la liberación del ámbito global, la ausencia de liberaciones dobles y de accesos inválidos, y la liberación correcta después de errores semánticos.

Problemas encontrados

Las pruebas de la Práctica 4 dejaron de pasar

Al cambiar la salida normal, la mayoría de las pruebas de la Práctica 4 fallaron: esperaban `AST:`, el árbol y `Programa sintacticamente correcto.`, pero el programa ahora imprime `Programa semanticamente correcto.`. Solo seguían pasando los casos con errores léxicos o sintácticos, cuya salida no cambió. No era un error del compilador sino una consecuencia esperada del enunciado. Lo resolvimos conservando la salida anterior bajo la bandera `-ast` y haciendo que los scripts y los objetivos del `Makefile` de las prácticas anteriores la usen.

Duplicado frente a fallo interno

Una primera idea es que la inserción devuelva solo "éxito" o "fallo". Con ese diseño, un `malloc` fallido durante el *rehash* se habría interpretado como una redeclaración. Por eso usamos tres resultados distintos (`ST_OK`, `ST_DUPLICATE`, `ST_INTERNAL_ERROR`), y establecimos que la tabla solo adquiere el símbolo cuando devuelve `ST_OK`.

Propiedad del símbolo en la inserción

Como la tabla no adquiere el símbolo cuando la inserción falla, `visit_variable_declaration` debe liberarlo en esa ruta. Olvidarlo habría producido una fuga justo en el caso de un error poco frecuente, que las pruebas normales no recorren.

Posición de la declaración

Los diagnósticos del enunciado esperan que una declaración se reporte en la posición del identificador y no en la de `int`. Revisamos `parse_declaration` y confirmamos que, desde la Práctica 4, el nodo ya almacenaba la posición del identificador, por lo que no fue necesario modificar el parser.

Límite por anidamiento profundo

Como en las prácticas anteriores, el recorrido recursivo consume pila en proporción al anidamiento. Una entrada con una cantidad extrema de bloques anidados puede agotarla. No agregamos un límite de profundidad y lo dejamos registrado como limitación conocida.

Conclusiones

Esta práctica nos mostró que el análisis semántico es, en gran parte, una cuestión de recordar lo que se ha visto mientras se recorre el árbol. El AST de la práctica anterior ya tenía toda la información necesaria (nombres, posiciones y estructura de bloques), así que no tuvimos que modificar el lexer, el parser ni el árbol: el cambio se concentró en cuatro módulos nuevos y en `main.c`. Eso confirma que separar el lexer, el parser y el AST nos permitió agregar una etapa nueva sin tocar las anteriores.

La pila de ámbitos resuelve de una sola vez la visibilidad, la salida de ámbito y el *shadowing*. La búsqueda avanza del ámbito más interno al global, por lo que siempre se encuentra la declaración más cercana, y al abandonar un bloque sus símbolos desaparecen sin ningún tratamiento adicional. La diferencia entre la búsqueda local, para declarar, y la búsqueda visible, para usar, es lo que permite al mismo tiempo prohibir la redeclaración en el mismo ámbito y permitirla en ámbitos distintos.

La parte más delicada fue, de nuevo, la propiedad de la memoria. Definir que la tabla adquiere el símbolo solo si la inserción termina con `ST_OK`, que el símbolo no es propietario del nodo del AST y que el analizador no es propietario del árbol nos obligó a razonar sobre cada ruta de error. También fue útil separar los errores semánticos, que se acumulan, de los fallos internos, que detienen el análisis de forma controlada. Con esa distinción, el compilador nunca presenta un

fallo de memoria como un error del programa fuente.

Las pruebas propias nos dieron una verificación independiente. Al generar los resultados esperados con un modelo de referencia distinto de la implementación en C, evitamos que un mismo error de comprensión estuviera en ambos lados. La tabla de símbolos conserva ya el tipo declarado de cada variable y la declaración original después de una redeclaración, por lo que la siguiente práctica podrá apoyarse en ella para inferir y verificar tipos. Como limitación, el diseño sigue siendo recursivo y la liberación inmediata de ámbitos tendrá que revisarse si el AST se anota con apuntadores a símbolos.

División del trabajo

Uso de herramientas de Inteligencia Artificial

References

[1] cppreference.com. (n.d.). C language. Disponible en https://en.cppreference.com/c/language

[2] cppreference.com. (n.d.). C11. Disponible en https://en.cppreference.com/c/11

[3] Free Software Foundation. (n.d.). GNU make. Disponible en https://www.gnu.org/software/make/manual/make.html